ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 3RD YEAR

MICROCONTROLLERS AND EMBEDDED SYSTEMS

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 3rd year Microcontrollers and Embedded Systems paper 1 · practice a.

Written by Academic Central for practice, following embedded systems modules at South African universities (e.g. University of Pretoria EMK 310 Microprocessors: architecture, memory and address decoding, embedded programming, I/O interfacing, timers, ADCs, serial ports and PWM). It is not a university memorandum.

How to mark: each ✓ is one mark. Give marks for correct steps even if a later step is wrong (consistent accuracy). The code shown is one correct answer; accept any working solution, including other register names for an equivalent microcontroller.

QUESTION 1

1.1 Harvard: separate memories and buses for program and data, so an instruction and data can be fetched at the same time ✓; von Neumann: one memory and bus for both ✓
Microcontroller: CPU, memory and peripherals (timers, ADC, I/O) on one chip, for control tasks ✓; microprocessor: CPU only, with external memory and peripherals, for general computing ✓ (4)

1.2 Flash: stores the program; kept without power ✓
SRAM: variables and the stack while the program runs; lost without power ✓
EEPROM: settings or calibration data that must survive a power cut ✓ (3)

1.3.1 0x8FFF − 0x8000 + 1 = 0x1000 ✓ = 4 096 bytes (4 KB) ✓ (2)

1.3.2 A15 = 1, A14 = 0, A13 = 0, A12 = 0 ✓✓ (A11 to A0 select the byte inside the peripheral) (2)

1.3.3 32 KB = 2^{15}, so 15 address lines ✓ (1)

1.4 (a) PORTB |= (1 << 5); (b) PORTB &= ~(1 << 2); (c) PORTB ^= (1 << 0); (d) if (PINB & (1 << 3))
Any two correct ✓, three correct ✓, all four correct ✓ (3)

[15]

QUESTION 2

2.1 The ISR can change the variable at any time; volatile tells the compiler to read it from memory every time instead of keeping a copy in a register ✓, so the main loop sees the new value ✓ (2)

2.2 Polling: the program checks the input repeatedly; simple, but uses CPU time and can miss short events ✓
Interrupt: the hardware calls an ISR when the event happens; fast response and the CPU is free otherwise ✓
Example: poll a slow menu button; use an interrupt for a wheel encoder or an emergency-stop input ✓ (3)

2.3 A mechanical switch's contacts bounce, making several on/off changes in a few milliseconds when it is pressed ✓
Software: after a change is detected, wait about 20 ms (or sample several times) and act only if the input is still in the new state ✓; act only on the change (edge), not on the level ✓ (3)

2.4 a = 44 ✓: 300 does not fit in 8 bits, so it wraps around: 300 − 256 = 44 ✓
b = −128 ✓: 127 is the largest int8_t, so adding 1 overflows to the most negative value (two's complement) ✓ (4)

2.5 PB5 as output ✓ PD2 as input with pull-up ✓ detect the press (pin reads 0) ✓ debounce delay ✓ toggle the LED ✓ wait for release so it toggles once per press ✓ (6)

```
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    DDRB |= (1 << PB5);      // LED pin as output
    DDRD &= ~(1 << PD2);     // button pin as input
    PORTD |= (1 << PD2);     // internal pull-up on
    while (1) {
        if (!(PIND & (1 << PD2))) {          // pressed (reads 0)
            _delay_ms(20);                      // debounce
            if (!(PIND & (1 << PD2))) {
                PORTB ^= (1 << PB5);         // toggle LED
                while (!(PIND & (1 << PD2)));  // wait for release
                _delay_ms(20);
            }
        }
    }
}
```

2.6 0b00010110 ✓ = 22 ✓ (2)

[20]

QUESTION 3

3.1.1 64 / 16 MHz ✓ = 4 µs ✓ (2)

3.1.2 10 ms / 4 µs = 2 500 ticks ✓✓ (3)
OCR1A = 2 500 − 1 = 2 499 ✓

3.2 Tick = 256 / 16 MHz = 16 µs ✓ (3)
Overflow = 256 × 16 µs ✓ = 4.10 ms ✓

3.3.1 f = f_clk / (N × 256) ✓ = 16 × 10^6 / (8 × 256) ✓ = 7 812.5 Hz ✓ (3)

3.3.2 Duty = (191 + 1)/256 ✓ = 75% ✓ (3)
V_avg = 0.75 × 5 = 3.75 V ✓

3.4.1 1 ms + (45/180) × 1 ms ✓ = 1.25 ms ✓ (2)

3.4.2 Tick = 8 / 16 MHz = 0.5 µs ✓ (2)
20 ms / 0.5 µs = 40 000, so ICR1 = 39 999 ✓

3.4.3 1.25 ms / 0.5 µs ✓ = 2 500 (accept 2 499) ✓ (2)

3.5 On an edge of the input, the hardware copies the running timer count into the capture register at once ✓; capturing the rising edge and then the falling edge, the difference × tick time is the pulse width ✓ (5)
Sound travels to the obstacle and back: d = v t / 2 ✓ = 343 × 1 160 × 10^{-6} / 2 ✓ = 0.20 m (19.89 cm) ✓

[25]

QUESTION 4

4.1.1 V = 154 × 5 / 1 024 ✓ = 0.75 V ✓ (3)
T = 0.752 V / 0.01 V/°C = 75.20 °C ✓

4.1.2 Resolution = 5 / 1 024 = 4.88 mV ✓ = 0.49 °C ✓ (2)

4.2 1.1 / 1 024 = 1.07 mV ✓ = 0.11 °C ✓ (3)
Maximum: 1.1 V / 10 mV = 110 °C ✓

4.3 A sample-and-hold keeps the input steady ✓. Starting with the most significant bit, each bit is set in turn, an internal DAC produces the matching voltage and a comparator checks whether it is above the input ✓; the bit is kept if the DAC voltage is lower and cleared if it is higher, until all bits are done (one bit per clock) ✓ (3)

4.4 A pin gives only about 20 to 40 mA at 5 V, while the motor needs 12 V and much more current; it would damage the pin ✓ (4)
Use a transistor or logic-level MOSFET switched by the pin (gate or base resistor) ✓, the motor between 12 V and the transistor ✓, and a flyback diode across the motor; for two directions, use an H-bridge driver IC ✓

4.5 Vout = 5 × R2 / (R1 + R2) ≤ 3.3 V gives R2 ≈ 3.5 kΩ; choose 3.3 kΩ ✓ (3)
Vout = 5 × 3.3 / (1.8 + 3.3) ✓ = 3.24 V ✓ (accept any suitable R2 with the correct calculation)

4.6 A timer that resets the microcontroller if the program stops running normally (hangs) ✓; the program must reset ("kick") the watchdog regularly to show it is still running ✓ (2)

[20]

QUESTION 5

5.1.1 1 start + 8 data + 1 stop = 10 bits ✓ (1)

5.1.2 9 600 / 10 ✓ = 960 bytes per second ✓ (2)

5.1.3 7 × 10 = 70 bits ✓; 70 / 9 600 = 7.29 ms ✓ (2)

5.2 UBRR = 16 × 10^6 / (16 × 9 600) − 1 = 103.17, so 103 ✓ (4)
Actual baud = 16 × 10^6 / (16 × 104) = 9 615.38 ✓
Error = (9 615.38 − 9 600) / 9 600 ✓ = 0.16% ✓

5.3 Wires: UART 2 (TX, RX); SPI 4 (MOSI, MISO, SCK, CS per device); I2C 2 (SDA, SCL) ✓✓ (6)
Clock: UART asynchronous (no clock line); SPI and I2C synchronous, clock from the master ✓
Selecting: UART point-to-point; SPI a chip-select line per device; I2C a 7-bit address in each message ✓
Speed: UART about 9.6 to 115.2 kbit/s typical; I2C 100 to 400 kbit/s; SPI several Mbit/s (fastest) ✓✓

5.4 Write: (0x68 << 1) | 0 = 0xD0 ✓ Read: 0xD1 ✓ (3)
I2C lines are open-drain: devices can only pull the line low, so pull-up resistors make the line high when no device pulls it down ✓

5.5 The master pulls CS low to select the one device it wants to talk to ✓; the other devices ignore the bus and keep their MISO output off, so several devices can share the same data and clock lines ✓ (2)

[20]

TOTAL: 100

END OF MARKING GUIDELINE